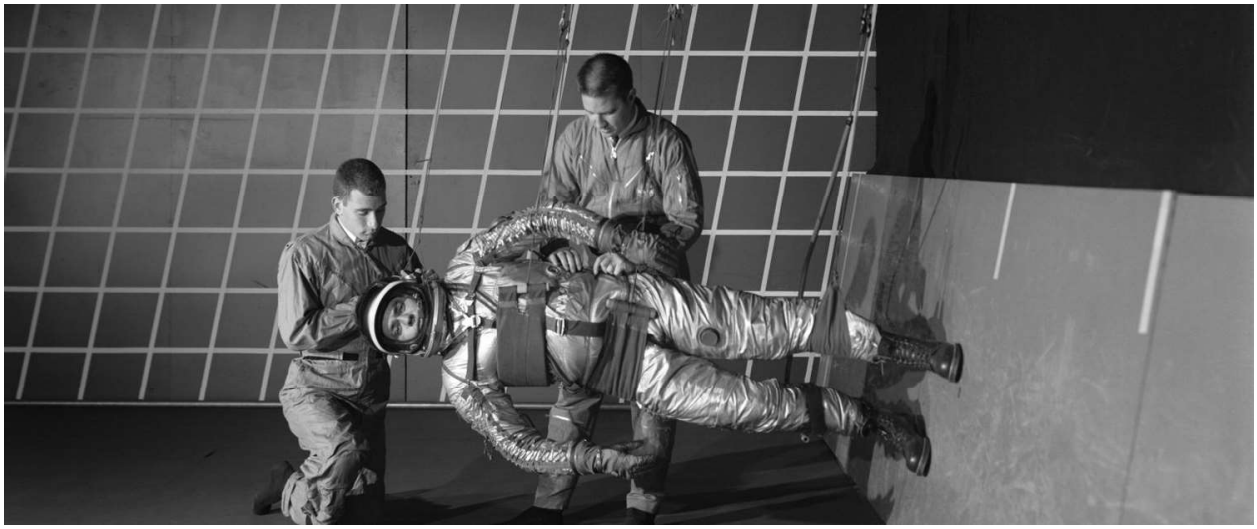


Linked list. (Part 2)

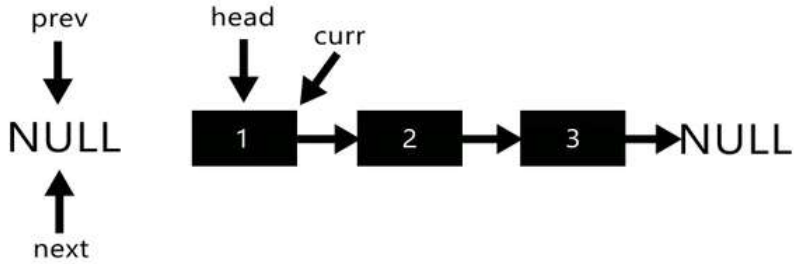
Debarka Sengupta, Ph.D., FNASc.

Content

Reversing a linked list
Adding two polynomials
Doubly linked list
Typical operations on DLL
Insert
Delete
Exercise
Reference

Reversing a linked list

Animation:




```
while (current != NULL)
    {
        next  = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head_ref = prev;
```

Source: Reverse a Linked List - GeeksforGeeks

C++ pseudocode

```
#include <iostream>

// Define a node structure for the singly linked list
struct Node {
    int data;
    Node* next;
};

// Function to reverse the singly linked list
Node* reverseLinkedList(Node* head) {
    Node* prev = nullptr; // Previous node initially points to nullptr
    Node* current = head; // Current node points to the head of the list
    Node* next = nullptr; // Next node initially points to nullptr

    // Traverse the list and reverse the links
    while (current != nullptr) {
        // Store the next node
        next = current->next;
        // Reverse the link
        current->next = prev;
        // Move pointers one step forward
        prev = current;
```

```
        current = next;
    }
    // At the end, prev will point to the new head of the reversed list
    return prev;
}

// Function to print the elements of the linked list
void printList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        std::cout << temp->data << " ";
        temp = temp->next;
    }
    std::cout << std::endl;
}

// Function to create a new node
Node* createNode(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = nullptr;
    return newNode;
}

int main() {
    // Create a sample linked list
    Node* head = createNode(1);
    head->next = createNode(2);
    head->next->next = createNode(3);
    head->next->next->next = createNode(4);

    std::cout << "Original list: ";
    printList(head);

    // Reverse the linked list
    head = reverseLinkedList(head);

    std::cout << "Reversed list: ";
    printList(head);

    return 0;
}
```

Explanation:

- We define a structure `Node` to represent each element of the singly linked list, containing integer data and a pointer to the next node.
- The `reverseLinkedList` function takes the head of the linked list as input and returns the new head of the reversed list.
- Inside the `reverseLinkedList` function, we initialize three pointers: `prev`, `current`, and `next`. `prev` initially points to `nullptr`, `current` points to the head of the list, and `next` is initialized to `nullptr`.

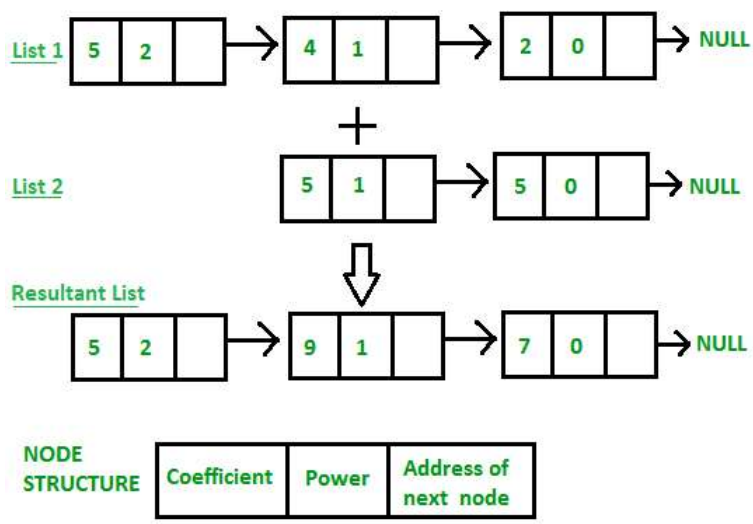
- We traverse the list using a `while` loop. In each iteration, we reverse the link of the `current` node to point to the `prev` node. Then, we move the `prev`, `current`, and `next` pointers one step forward.
- After traversing the entire list, `prev` will point to the new head of the reversed list, so we return `prev`.
- In the `main` function, we create a sample linked list and print it.
- Then, we call the `reverseLinkedList` function to reverse the list and print the reversed list.

Adding two polynomials

Polynomial addition in c programming is a fundamental operation in mathematics and computer science, widely used in various applications such as signal processing, computer graphics, and scientific simulations.

Example:

```
Input:
     1st number = 5x2 + 4x1 + 2x0
     2nd number = -5x1 - 5x0
Output:
        5x2-1x1-3x0
Input:
     1st number = 5x3 + 4x2 + 2x0
     2nd number = 5x^1 - 5x^0
Output:
        5x3 + 4x2 + 5x1 - 3x0
```



Pseudocode:

```
// Define a structure for the polynomial term
struct Term {
    int coefficient;
    int exponent;
    Term* next;
};
```

```
// Function to insert a term into the polynomial
void insertTerm(Term*& head, int coefficient, int exponent) {
    // Create a new term
    Term* newTerm = new Term();
    newTerm->coefficient = coefficient;
    newTerm->exponent = exponent;
    newTerm->next = nullptr;

    // If the polynomial is empty, make the new term as the head
    // Otherwise, insert the term at the end of the polynomial
}

// Function to add two polynomials represented by linked lists
Term* addPolynomials(Term* poly1, Term* poly2) {
    // Create a new polynomial for the result
    Term* result = nullptr;

    // Initialize pointers to traverse both polynomials
    Term* current1 = poly1;
    Term* current2 = poly2;

    // Traverse both polynomials simultaneously
    while (current1 != nullptr && current2 != nullptr) {
        // If the exponents of the current terms are equal, add their coefficients and in
sert the result into the new polynomial
        // If the exponent of the current term in poly1 is greater than the exponent of t
he current term in poly2, insert the current term from poly1 into the result
        // If the exponent of the current term in poly2 is greater than the exponent of t
he current term in poly1, insert the current term from poly2 into the result
        // Move to the next term in the polynomial with the smaller exponent
    }

    // Copy remaining terms of poly1, if any, into the result polynomial
    // Copy remaining terms of poly2, if any, into the result polynomial

    // Return the result polynomial
}

// Function to display the polynomial
void displayPolynomial(Term* poly) {
    // Traverse the polynomial and display each term
}
```

Code:

```
#include <iostream>

using namespace std;

// Define a structure for the polynomial term
```

```
struct Term {
    int coefficient;
    int exponent;
    Term* next;
};

// Function to insert a term into the polynomial
void insertTerm(Term*& head, int coefficient, int exponent) {
    // Create a new term
    Term* newTerm = new Term();
    newTerm->coefficient = coefficient;
    newTerm->exponent = exponent;
    newTerm->next = nullptr;

    // If the polynomial is empty, make the new term as the head
    if (head == nullptr) {
        head = newTerm;
    } else {
        // Otherwise, insert the term at the end of the polynomial
        Term* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newTerm;
    }
}

// Function to add two polynomials represented by linked lists
Term* addPolynomials(Term* poly1, Term* poly2) {
    // Create a new polynomial for the result
    Term* result = nullptr;

    // Initialize pointers to traverse both polynomials
    Term* current1 = poly1;
    Term* current2 = poly2;

    // Traverse both polynomials simultaneously
    while (current1 != nullptr && current2 != nullptr) {
        // If the exponents of the current terms are equal, add their coefficients and in
sert the result into the new polynomial
        if (current1->exponent == current2->exponent) {
            int sumCoefficients = current1->coefficient + current2->coefficient;
            insertTerm(result, sumCoefficients, current1->exponent);
            current1 = current1->next;
            current2 = current2->next;
        }
        // If the exponent of the current term in poly1 is greater than the exponent of t
he current term in poly2, insert the current term from poly1 into the result
        else if (current1->exponent > current2->exponent) {
            insertTerm(result, current1->coefficient, current1->exponent);
            current1 = current1->next;
```

```
        }
        // If the exponent of the current term in poly2 is greater than the exponent of t
he current term in poly1, insert the current term from poly2 into the result
        else {
            insertTerm(result, current2->coefficient, current2->exponent);
            current2 = current2->next;
        }
    }

    // Copy remaining terms of poly1, if any, into the result polynomial
    while (current1 != nullptr) {
        insertTerm(result, current1->coefficient, current1->exponent);
        current1 = current1->next;
    }

    // Copy remaining terms of poly2, if any, into the result polynomial
    while (current2 != nullptr) {
        insertTerm(result, current2->coefficient, current2->exponent);
        current2 = current2->next;
    }

    // Return the result polynomial
    return result;
}

// Function to display the polynomial
void displayPolynomial(Term* poly) {
    if (poly == nullptr) {
        cout << "0";
        return;
    }

    Term* temp = poly;
    while (temp != nullptr) {
        cout << temp->coefficient;
        if (temp->exponent != 0) {
            cout << "x^" << temp->exponent;
        }
        if (temp->next != nullptr && temp->next->coefficient >= 0) {
            cout << " + ";
        } else if (temp->next != nullptr) {
            cout << " ";
        }
        temp = temp->next;
    }
}

// Function to delete the linked list
void deleteList(Term*& head) {
    while (head != nullptr) {
        Term* temp = head;
```

```
        head = head->next;
        delete temp;
    }
}

int main() {
    // Create two polynomials
    Term* poly1 = nullptr;
    insertTerm(poly1, 5, 3);
    insertTerm(poly1, 4, 2);
    insertTerm(poly1, 2, 0);

    Term* poly2 = nullptr;
    insertTerm(poly2, 3, 4);
    insertTerm(poly2, -1, 2);
    insertTerm(poly2, 2, 1);

    // Display the polynomials
    cout << "Polynomial 1: ";
    displayPolynomial(poly1);
    cout << endl;

    cout << "Polynomial 2: ";
    displayPolynomial(poly2);
    cout << endl;

    // Add the polynomials and display the result
    Term* result = addPolynomials(poly1, poly2);
    cout << "Sum: ";
    displayPolynomial(result);
    cout << endl;

    // Delete the linked lists
    deleteList(poly1);
    deleteList(poly2);
    deleteList(result);

    return 0;
}
```

Output:

Polynomial 1: 5x^3 + 4x^2 + 2
Polynomial 2: 3x^4 -1x^2 + 2x^1
Sum: 3x^4 + 5x^3 + 3x^2 + 2x^1 + 2

Why linked lists are preferred for polynomial addition?

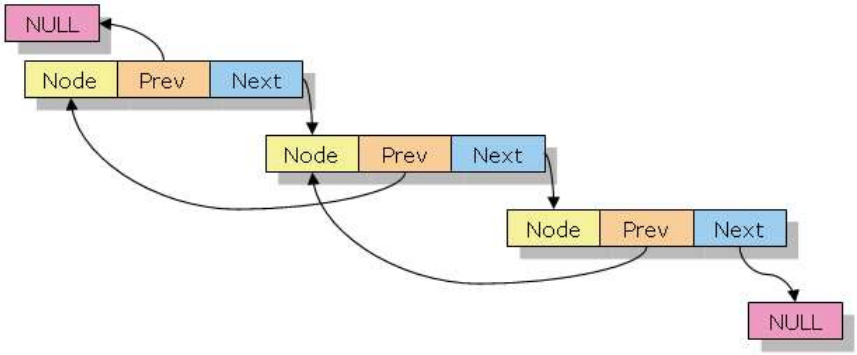
- Linked lists are preferred for adding two polynomials over arrays due to their dynamic memory allocation and efficient insertion and deletion capabilities.

- Adding polynomials with linked lists involves creating a new linked list of size **m + n** for storing the result, offering a simple and concise way to modify existing nodes rather than creating new ones.
- Linked lists efficiently handle the storage and retrieval of polynomial coefficients and degrees, making them suitable for polynomial addition.
- They can handle sparse polynomials effectively by storing only non-zero terms, avoiding memory wastage.
- In contrast, arrays have a fixed size, making them less suitable for handling polynomials of varying degrees and less efficient for insertion and deletion operations, as they may require shifting elements.
- Therefore, linked lists are preferred for polynomial addition due to their flexibility and efficiency in handling the dynamic nature of polynomial terms.

Doubly linked list

Applications of Doubly Linked List:

- Web browsers use it for backward and forward navigation of web pages.
- LRU (Least Recently Used) / MRU (Most Recently Used) Cache are constructed using Doubly Linked Lists.
- Various applications utilize it to maintain undo and redo functionalities.
- In Operating Systems, a doubly linked list is maintained by thread schedulers to track processes being executed.



Depiction of a doubly linked list.

Node in a doubly linked list:

```
struct Node {
    int data;
    Node* prev;
    Node* next;
};
```

In this struct:

- `data` stores the value or data associated with the node.
- `prev` is a pointer to the previous node in the doubly linked list.
- `next` is a pointer to the next node in the doubly linked list.

Each node in a doubly linked list contains pointers to both the previous and next nodes, allowing for traversal in both directions.

Typical operations on DLL

1. **Insertion Operations:**
   - **Insertion at the Beginning:** Inserting a new node at the beginning of the doubly linked list.
   - **Insertion at the End:** Inserting a new node at the end of the doubly linked list.
   - **Insertion at a Given Position:** Inserting a new node at a specified position within the doubly linked list.
2. **Deletion Operations:**
   - **Deletion at the Beginning:** Deleting the first node of the doubly linked list.
   - **Deletion at the End:** Deleting the last node of the doubly linked list.
   - **Deletion at a Given Position:** Deleting a node at a specified position within the doubly linked list.
   - **Deletion by Value:** Deleting a node with a given value from the doubly linked list.
3. **Traversal Operations:**
   - **Forward Traversal:** Traversing the doubly linked list from the beginning to the end.
   - **Reverse Traversal:** Traversing the doubly linked list from the end to the beginning.
4. **Search Operations:**
   - **Search by Value:** Searching for a node with a specific value within the doubly linked list.
   - **Search by Position:** Searching for a node at a specified position within the doubly linked list.
5. **Updating Operations:**
   - **Update Value:** Updating the value of a node at a specified position within the doubly linked list.
6. **Miscellaneous Operations:**
   - **Get Length:** Determining the number of nodes in the doubly linked list.
   - **Concatenation:** Concatenating two doubly linked lists.
   - **Reversal:** Reversing the order of nodes in the doubly linked list.

Insert

C++ pseudocode

```
#include <iostream>

using namespace std;

// Define a struct for a node in the doubly linked list
struct Node {
    int data;
    Node* prev;
    Node* next;
};

// Function to insert a new node at the end of the doubly linked list
void insertAtEnd(Node*& head, int newData) {
    // Create a new node
```

```
    Node* newNode = new Node();
    newNode->data = newData;
    newNode->next = nullptr;

    // If the list is empty, make the new node the head
    if (head == nullptr) {
        newNode->prev = nullptr;
        head = newNode;
        return;
    }

    // Traverse the list to find the last node
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }

    // Insert the new node after the last node
    temp->next = newNode;
    newNode->prev = temp;
}

// Function to display the doubly linked list
void displayList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Initialize an empty doubly linked list
    Node* head = nullptr;

    // Insert nodes at the end
    insertAtEnd(head, 1);
    insertAtEnd(head, 2);
    insertAtEnd(head, 3);

    // Display the doubly linked list
    cout << "Doubly linked list after insertion of 3 nodes: ";
    displayList(head);

    return 0;
}
```

Explanation:

1. **Node Struct:**

- We define a `Node` struct to represent a node in the doubly linked list. Each node contains three fields:
  - `data`: An integer to store the data associated with the node.
  - `prev`: A pointer to the previous node in the doubly linked list.
  - `next`: A pointer to the next node in the doubly linked list.

2. **Insertion Function (`insertAtEnd`):**
   - This function inserts a new node with a given data value at the end of the doubly linked list.
   - If the list is empty (i.e., `head` is `nullptr`), the new node becomes the head of the list.
   - If the list is not empty, the function traverses the list to find the last node.
   - Once the last node is found, the new node is inserted after it by updating the `next` pointer of the last node to point to the new node and the `prev` pointer of the new node to point to the last node.
3. **Display Function (`displayList`):**
   - This function traverses the doubly linked list from the head to the end, printing the data value of each node.
4. **Main Function:**
   - We initialize an empty doubly linked list by setting the `head` pointer to `nullptr`.
   - We insert three nodes with data values 1, 2, and 3 into the doubly linked list using the `insertAtEnd` function.
   - Finally, we display the contents of the doubly linked list using the `displayList` function.

Insert at the beginning C++ pseudocode:

```
void insertAtBeginning(Node*& head, int newData) {
    // Create a new node
    Node* newNode = new Node();
    newNode->data = newData;
    newNode->prev = nullptr;

    // If the list is not empty, update the prev pointer of the current head
    if (head != nullptr) {
        head->prev = newNode;
    }

    // Update the next pointer of the new node to point to the current head
    newNode->next = head;

    // Update the head pointer to point to the new node
    head = newNode;
}
```

Delete

```
#include <iostream>

using namespace std;

// Define a struct for a node in the doubly linked list
```

```
struct Node {
    int data;
    Node* prev;
    Node* next;
};

// Function to insert a new node at the end of the doubly linked list
void insertAtEnd(Node*& head, int newData) {
    // Create a new node
    Node* newNode = new Node();
    newNode->data = newData;
    newNode->next = nullptr;

    // If the list is empty, make the new node the head
    if (head == nullptr) {
        newNode->prev = nullptr;
        head = newNode;
        return;
    }

    // Traverse the list to find the last node
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }

    // Insert the new node after the last node
    temp->next = newNode;
    newNode->prev = temp;
}

// Function to delete a node with a given value from the doubly linked list
void deleteNode(Node*& head, int key) {
    // If the list is empty, return
    if (head == nullptr) {
        return;
    }

    // Find the node with the given value
    Node* curr = head;
    while (curr != nullptr && curr->data != key) {
        curr = curr->next;
    }

    // If the node with the given value is not found, return
    if (curr == nullptr) {
        return;
    }

    // If the node to be deleted is the head node
    if (curr == head) {
```

```
        head = curr->next;
    }

    // Update the next pointer of the previous node
    if (curr->prev != nullptr) {
        curr->prev->next = curr->next;
    }

    // Update the prev pointer of the next node
    if (curr->next != nullptr) {
        curr->next->prev = curr->prev;
    }

    // Delete the node
    delete curr;
}

// Function to display the doubly linked list
void displayList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Initialize an empty doubly linked list
    Node* head = nullptr;

    // Insert nodes at the end
    insertAtEnd(head, 1);
    insertAtEnd(head, 2);
    insertAtEnd(head, 3);

    // Display the doubly linked list before deletion
    cout << "Doubly linked list before deletion: ";
    displayList(head);

    // Delete node with value 2
    deleteNode(head, 2);

    // Display the doubly linked list after deletion
    cout << "Doubly linked list after deletion: ";
    displayList(head);

    return 0;
}
```

Explanation:

In the provided C++ code, a `Node` struct is defined to represent a node in a doubly linked list. Each node encapsulates three fields: `data`, an integer to store the associated data; `prev`, a pointer to the previous node in the list; and `next`, a pointer to the next node in the list. The `insertAtEnd` function inserts a new node with a given data value at the end of the doubly linked list. The `deleteNode` function deletes a node with a specified value from the list. It traverses the list to locate the node with the given value, and upon finding it, it addresses three cases: if the node to be deleted is the head node, it updates the head pointer; it then adjusts the pointers of the neighboring nodes to bypass the node to be deleted; finally, it deallocates the memory occupied by the node. The `displayList` function facilitates the traversal of the doubly linked list from the head to the end, printing the data value of each node encountered. In the `main` function, an empty doubly linked list is initialized, three nodes with data values of 1, 2, and 3 are inserted using the `insertAtEnd` function, and the contents of the list are displayed before and after the deletion of a node with the value 2 using the `deleteNode` function. This code exemplifies the operations involved in managing a doubly linked list in C++.

Exercise

1. **Palindrome Check:**
   Given a doubly linked list, determine whether it is a palindrome. A palindrome is a sequence that reads the same forwards and backwards. For example, the list 1 ↔ 2 ↔ 3 ↔ 2 ↔ 1 is a palindrome, while the list 1 ↔ 2 ↔ 3 is not.

2. **Intersection of Two Doubly Linked Lists:**
   Given two doubly linked lists, determine if the two lists intersect and return the intersecting node. If the lists do not intersect, return
   `nullptr`. Intersection is defined based on reference, not value.

Reference

1. Practice problems: Top 50 Problems on Linked List Data Structure asked in SDE Interviews - GeeksforGeeks